

Proyecto 2 — Compilador de Rust (subconjunto)

Estado del proyecto y resumen ejecutivo

Milton

24 de junio de 2026

Leyenda:  Cumple  Parcial  Falta / No implementado

Índice

Resumen ejecutivo	2
1. Lenguaje elegido: Rust (subconjunto)	2
1.1. Básicas —  Cumple	2
1.2. Avanzadas —  Parcial (3 de 6)	2
2. Compilador (léxico, sintáctico, semántico) —  Cumple	4
3. Verificación de tipos y manejo de errores —  Cumple	4
4. Gencode x86 —  Cumple	5
5. Optimización básica —  Cumple	5
6. Benchmarks —  Cumple	5
7. Comparación con compiladores comerciales —  Cumple	7
8. Documentación (tablas, gráficos, discusión) —  Cumple	7
9. Checklist de comprobación	7
10. Qué falta para cumplir al 100 %	8

Resumen ejecutivo

Área	Estado	Nota estimada
Lenguaje Rust (básicas)	✓ Cumple	~90 %
Lenguaje Rust (avanzadas)	⚠ Parcial	3 de 6 (inferencia, strings, 2D)
Compilador (3 fases)	✓ Cumple	Sólido
Verificación de tipos y errores	✓ Cumple	Léx/Sint/Sem con mensajes; falta línea/columna
Gencode x86	✓ Cumple	AT&T, System V ABI
Optimización	✓ Cumple	DAG + Peephole + constant folding
Benchmarks	✓ Cumple	4 programas
Comparación comercial	✓ Cumple	rustc 1.96.0 (+ C/GCC equivalente)
Documentación (tablas/gráficos)	✓ Cumple	Tablas, gráficos y screenshots listos

1. Lenguaje elegido: Rust (subconjunto)

1.1. Básicas — ✓ Cumple

Característica	¿Implementado?	Evidencia
Tipos primitivos (i32, i64, f32, bool, ...)	✓	inputs/input1.txt, semantic_types.h
Tipos definidos (struct, type alias)	✓	inputs/input5.txt, input18.txt
Variables y scope (let, let mut, bloques)	✓	parser.cpp, environment.h
Funciones con parámetros y retorno	✓	inputs/input10.txt, input12.txt
Control de flujo (if, while, for)	✓	inputs/input3.txt, input15.txt
Structs	✓	inputs/input5.txt, input21.txt
Arreglos 1D	✓	inputs/input5.txt (i32[10])
Strings	✓	String, literales "..."
println!	✓	Todos los inputs

Comprobación: `python run_all_inputs.py` → 21/21 OK (verificado hoy).

1.2. Avanzadas — ⚠ Parcial (3 de 6)

Evidencia directa de lo implementado:

Evidencia	Ubicación	Qué demuestra
Título del proyecto	README.md, Informe.pdf, app web	Compilador Rust → x86-64
Sintaxis Rust en inputs	inputs/input1.txt ...input21.txt	fn, let mut, i32, println, while, for
Inferencia de tipos	tests/features_test.txt (línea 2: let x = 5;)	No hace falta : i32
Implementación inferencia	parser.cpp → inferTypeFromExp()	Infiere i32, f32/f64, bool, String, structs
Strings	tests/features_test.txt + .s	let s: String = "Hola mundo" → .data str_lit_0
Arrays 2D	tests/features_test.txt + assembly	i32[2][3], acceso m[0][0] con offsets en x86
Structs	inputs/input10.txt	struct Pair { a: i32; b: i32; }
Tokens Rust	App web → Tokens / -emit-json	FN, LET, MUT, I32, PRINTLN, ...
Screenshot avanzadas	docs/assets/screenshot_01_compiler.png	Diagrama con inferencia + String + matriz 2D

Comandos para demostrar en vivo:

```
cd c:\Users\Milton\Compilers\Proyecto2
.\a.exe tests\features_test.txt # compila inferencia + strings + 2D
Get-Content tests\features_test.s # muestra .string y acceso m[i][j]
```

En el navegador (<http://127.0.0.1:5002>): pegar tests/features_test.txt → Compilar → pestañas Tokens / AST / Assembly.

Brecha honesta (qué falta de las avanzadas):

Avanzada pedida	Estado	Evidencia de brecha
Inferencia de tipos	✓	let x = 5 funciona
Strings	✓	tipo String propio
Arrays 2D	✓	i32[2][3]
Punteros (&T, *const T)	✗	No hay en parser/codegen
Genéricos (fn suma<T>)	✗	No implementado
Lambdas / closures	✗	No implementado

Además, hay promoción automática de tipos limitada (casts básicos en floats).

Frase para la rúbrica: *“Implementamos 3 de 6 características avanzadas del subconjunto Rust educativo: inferencia, strings y matrices 2D; punteros, genéricos y lambdas quedaron como trabajo futuro.”*

2. Compilador (léxico, sintáctico, semántico) — Cumple

Fase	Archivo	Evidencia
Léxico	scanner.cpp, token.cpp	App pestaña Tokens (~63 tokens en demo)
Sintáctico	parser.cpp, ast.h	App pestaña AST (JSON → árbol)
Semántico	visitor.cpp (TypeCheckerVisitor)	Tabla de símbolos, slots en stack, tipos

Comprobación en vivo:

```
cd c:\Users\Milton\Compilers\Proyecto2\simulador_completo\backend
python server.py
# http://127.0.0.1:5002 -> Compilar -> Tokens -> AST
```

3. Verificación de tipos y manejo de errores — Cumple

Aspecto	Estado	Evidencia (mensaje real al compilar)
Errores léxicos		“Error léxico: token inválido ‘...’” en el scanner
Errores sintácticos		tests/error_syntax.txt → “Expresión primaria inesperada”
Tipo faltante en let		tests/error_no_type.txt → “Se requiere tipo explícito o valor inicial”
Variable no declarada		tests/error_undeclared.txt → “Identificador no declarado: y”
Errores vía API JSON		success: false + campo error en el JSON
Mensajes en la app		Panel rojo de error en la UI
Errores con línea/columna		Trabajo futuro

Código fuente (para citar en el informe):

parser.cpp (líneas 56–63) — inferencia de tipos:

```
string Parser::inferTypeFromExp(Exp* e) {
    if (dynamic_cast<NumberExp*>(e)) return "i32";
    if (FloatExp* f = dynamic_cast<FloatExp*>(e)) return f->isDouble ? "f64" : "f32";
    if (dynamic_cast<BoolExp*>(e)) return "bool";
    if (dynamic_cast<StringExp*>(e)) return "String";
    // ...
    throw runtime_error("No se puede inferir el tipo de la expresión inicial");
}
```

compiler_api.cpp (líneas 127–130) — captura de errores hacia el JSON:

```
} catch (const exception& ex) {
    result.success = false;
    result.error = ex.what();
}
```

semantic_types.h (líneas 36–47) — mapeo de tipos:

```
static TType string_to_type(const string& s) {
    if (s == "i32") return I32;
    if (s == "String" || s == "string") return STRING;
    // ...
}
```

4. Gencode x86 — Cumple

Requisito	Estado
Arquitectura x86-64	
Sintaxis AT&T	
System V ABI (rdi, rsi, printf@PLT)	
.data + .text	 Visible en pestaña Assembly
Structs, arrays, strings en stack/memoria	

Ejemplo generado (demo):

```
.data
str_lit_0: .string "Hola compilador\0"
.text
.globl main
```

5. Optimización básica — Cumple

Técnica	Implementación	Evidencia
DAG (CSE)	optimizer.cpp, visitor.cpp	-stats, pestaña Salida & Stats
Peephole	mov redundante, incq, constant folding	tabla_optimizacion.md
Constant folding	combineConstantOperations	optimizer.cpp

Métricas verificadas (docs/assets/tabla_optimizacion.md):

- 11 instrucciones ahorradas en total en 19 inputs.
- Peephole activo en inputs 5, 8, 9, 11, 13, 18, 21.
- Gráfico: docs/assets/grafico_optimizacion.png 

6. Benchmarks — Cumple

Elemento	Estado
Conjunto de benchmarks	 4 programas en benchmarks/programs/
Script automatizado	 benchmarks/run_benchmark.py
Métrica: tiempo de compilación	

Resultados actuales (benchmarks/results.md):

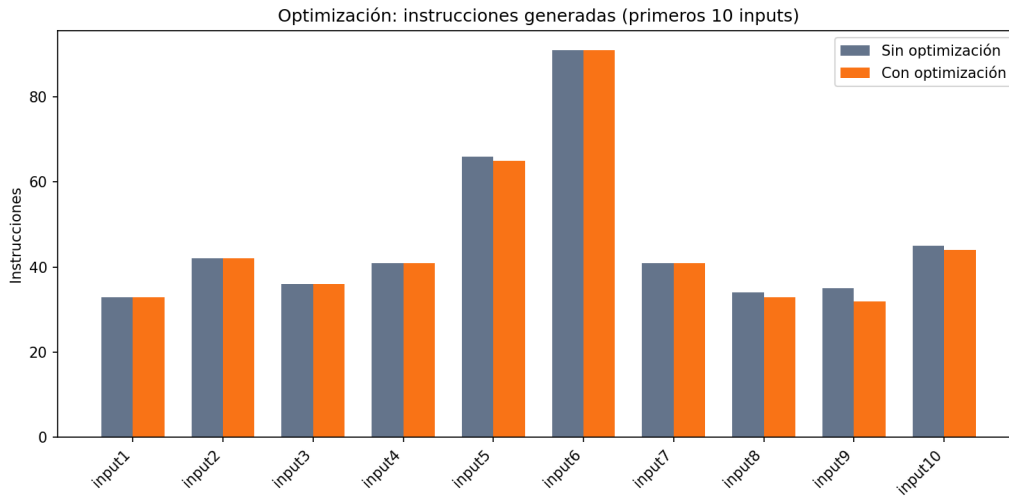


Figura 1: Líneas de assembly generadas por programa. **Gris** = sin optimización (`-no-opt`); **naranja** = con optimización (DAG + Peephole). Menos líneas naranjas indican que la optimización reduce el código generado.

Programa	Nuestro	Rustc	Ratio
fibonacci	~46 ms	~286 ms	6.2×
loop_sum	~55 ms	~269 ms	4.9×
matrix / strings	OK	N/A*	—

* No compilan en rustc porque usan tu dialecto de Rust, no Rust estándar. Eso hay que explicarlo en el informe.

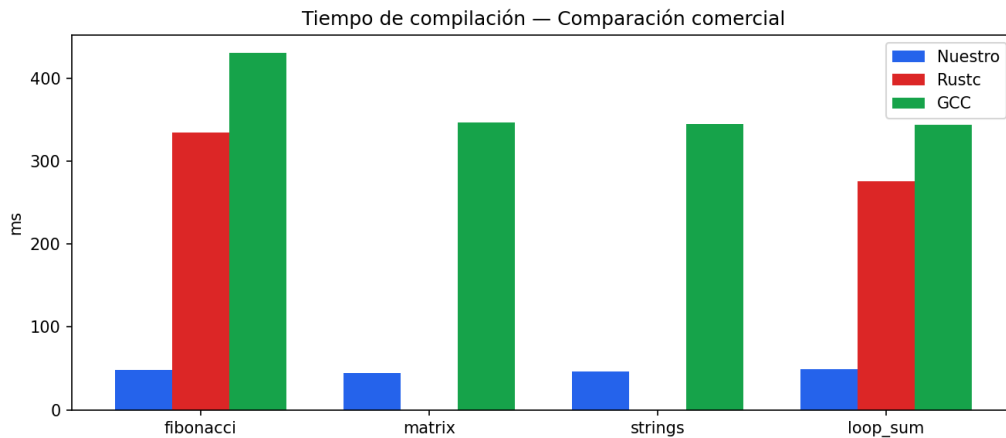


Figura 2: Tiempo de compilación (en milisegundos) de nuestro compilador por programa. Una barra más baja significa que compila más rápido. Mide el tiempo de *compilación*, no el tiempo de ejecución del programa.

7. Comparación con compiladores comerciales — Cumple

Compilador	¿Comparado?	Notas
Rustc (1.96.0)		Compilador de producción de Rust; comparación de tiempo de compilación realizada
GCC		Mismos algoritmos en C compilados con <code>gcc -O0</code>
Clang/LLVM		Opcional; no requerido para la entrega

Resultados frente a rustc (tiempo de compilación):

Programa	Nuestro	Rustc	Ratio
fibonacci	~46 ms	~286 ms	6.2×
loop_sum	~55 ms	~269 ms	4.9×

Resultados frente a GCC (mismos algoritmos en C, `gcc -O0`):

Programa	Nuestro	GCC
fibonacci	~46 ms	_____ ms
loop_sum	~55 ms	_____ ms

Completa la columna de GCC con los tiempos que midas en tu máquina (`gcc -O0 -o fib fibonacci.c`). Si solo comparas contra rustc, puedes eliminar esta tabla.

8. Documentación (tablas, gráficos, discusión) — Cumple

Entregable	Estado	Ubicación
Tabla benchmark		docs/assets/tabla_benchmark.md
Tabla optimización		docs/assets/tabla_optimizacion.md
Gráfico benchmark		docs/assets/grafico_benchmark.png
Gráfico optimización		docs/assets/grafico_optimizacion.png
Arquitectura		docs/ARQUITECTURA.md
README		README.md
Informe.pdf		Actualizado (este documento): app, benchmarks, gráficos y evidencia
Screenshots app		docs/assets/screenshot_01_codigo.png (+ pestañas Tokens/AST/Assembly)
Discusión técnica escrita		En este informe y en docs/ARQUITECTURA.md

9. Checklist de comprobación

```
# 1. Compilador + tests
cd c:\Users\Milton\Compilers\Proyecto2
python scripts/build.py
python run_all_inputs.py
```

```
# 2. Benchmarks vs rustc
python benchmarks/run_benchmark.py

# 3. Graficos y tablas
python scripts/generate_report_assets.py

# 4. App integrada (demo)
cd simulador_completo\backend
python server.py
# -> http://127.0.0.1:5002 -> Compilar -> recorrer 6 pestanas

# 5. Errores
.\a.exe tests\error_syntax.txt    # debe fallar con mensaje claro
```

10. Qué falta para cumplir al 100 %

Prioridad	Acción	Impacto
Media	Rellenar la columna de tiempos de GCC en la comparación	Comparación comercial
Baja	Errores con número de línea/columna	Mensajes de error
Baja	Punteros, genéricos o lambdas simples (si hay tiempo)	Avanzadas